

# COE025 NLP — Week 2: Text Classification

Asst. Prof. Yiğit Bekir Kaya — Istinye University, Fall 2026 — exam summary of all slides (1-32)

How to use this file: every topic has (1) the key ideas, (2) the important terms, (3) a worked example, (4) exam traps, and (5) a short Arabic note. The last pages have a formula sheet, a glossary, and a self-test with answers.

## The Week in One Picture

TEXT CLASSIFICATION:  $f(\text{text}) \rightarrow \text{label}$

|        |           |       |         |        |            |            |
|--------|-----------|-------|---------|--------|------------|------------|
| Tasks: | Sentiment | Spam  | Topic   | Intent | Moderation | Authorship |
|        | Medical   | Legal | Emotion |        |            |            |

[illegible]

Models:    Naive Bayes (FAST) | Logistic Regression (BALANCED) | SVM (ACCURATE)

Features: Statistical | Lexical | Linguistic | Domain-specific

Project 1: NB + one other method, team of 5, present in Week 4

| Topic | Slides | Subject                                            |
|-------|--------|----------------------------------------------------|
| 1     | 3-6    | What text classification is; binary vs multi-class |
| 2     | 7-11   | The tasks: sentiment, spam, topic, others          |
| 3     | 12-14  | ML pipeline; why classical methods                 |
| 4     | 15-16  | Naive Bayes                                        |
| 5     | 18-20  | Logistic Regression; NB vs LR                      |
| 6     | 22     | Support Vector Machines (SVM)                      |
| 7     | 24     | Feature engineering                                |
| 8     | 25-31  | Comparing methods; Project 1; takeaways            |

## Topic 1 — What Is Text Classification? (Slides 3-6)

### Recap from Week 1 (slide 4)

- **Text → Numbers:** computers only work with numbers. Tools: **tokenization** (cut text into pieces called tokens), **subwords / BPE** (cut words into smaller pieces, e.g. "playing" = "play" + "ing"), **embeddings** (turn each word into a list of numbers).
- **The Model Zoo:** Classical ML → Neural → Transformers. Week 2 = **classical ML** only.
- **The Bulldozer Problem:** do not use LLMs (Large Language Models, like ChatGPT) for everything. A simple problem needs a simple tool.
- **Four Projects:** Classification, Sequence, Speech, MT (Machine Translation). Week 2 starts Project 1.

### Plan of the lecture (slide 5)

Block 1: the problems (what is text classification, applications). Block 2: Naive Bayes, Logistic Regression. Block 3: SVM, feature engineering, the project.

### Definition (slide 6)

**Text classification = assigning predefined categories/labels to text documents.**

- **Label / category / class:** the name of a group (e.g. "Spam", "Sports").
- **Predefined:** the list of labels is fixed **before**. The model **chooses** from the list; it never invents a new label.
- **Document:** any piece of text (email, tweet, review, article).

### The formula

```
f(text) -> label  
f("This movie was amazing!") -> Positive
```

f is the classifier (a function): input = text, output = one label.

### Two types

| Type                       | Number of labels | Slide example                    |
|----------------------------|------------------|----------------------------------|
| Binary classification      | exactly 2        | Spam vs Not Spam                 |
| Multi-class classification | more than 2      | Topic: Sports, Politics, Tech... |

**Exam traps:** (1) The type depends only on the **number of possible labels**, not on the text. (2) If the output is **new text** (a summary, a translation) and not a label from a fixed list, it is **not** text classification.

التصنيف النصي هو أن نعطي النص label واحدا من قائمة محددة مسبقا (predefined). النموذج لا ي اخترع تصنيفا جديدا. إذا كان هناك خياران فقط فهو binary، وإذا كان أكثر من خيارين فهو multi-class.

## Topic 2 — The Text Classification Tasks (Slides 7-11)

All tasks follow the same rule  $f(\text{text}) \rightarrow \text{label}$ . Only the **list of labels** changes.

### Sentiment Analysis (slide 7) — "Understanding Opinions & Emotions"

Decide if a text is **Positive, Negative, or Neutral** (3 labels → multi-class). **Sentiment** = the feeling or opinion in a text.

| Label    | Example                                      |
|----------|----------------------------------------------|
| Positive | "Love this product!"                         |
| Neutral  | "The item arrived" (only a fact, no feeling) |
| Negative | "Terrible experience"                        |

- **Uses:** customer feedback analysis, brand monitoring, product reviews, social media monitoring, market research.
- **Challenges:** **sarcasm** (words say the opposite of the meaning: "Great, my phone broke again" = Negative), **context-dependent meaning** (same word, different meaning in different situations), **domain-specific language** (each field has its own words; domain = field).

### Spam Detection (slide 8) — "Filtering Unwanted Messages"

**Binary:** **Spam** (unwanted) vs **Ham** (legitimate = real, wanted messages).

|                  | Spam                                                                      | Ham                                                         |
|------------------|---------------------------------------------------------------------------|-------------------------------------------------------------|
| Examples         | "CONGRATULATIONS! You've WON \$1000000!!!" / "Click HERE for FREE iPhone" | "Meeting tomorrow at 3pm" / "Your Amazon order has shipped" |
| Features (clues) | ALL CAPS, excessive punctuation, urgency, offers                          | normal language, relevant content, known sender             |

- **Uses:** email filtering, SMS filtering, comment moderation, user safety.

### Topic Classification (slide 9) — "Organizing Content by Subject"

**Multi-class:** assign documents to topical categories.

| Label      | Example                                  |
|------------|------------------------------------------|
| Sports     | "The Lakers won 108-102 in overtime..."  |
| Technology | "Apple announced new iPhone features..." |
| Business   | "Stock markets closed higher today..."   |
| Politics   | "Congress passed the new bill..."        |

- **Uses:** news aggregation (collecting news from many sources), document organization, content recommendation, search engines.

### Other Text Classification Problems (slide 10)

| Task                          | What it does                                 | Example                               |
|-------------------------------|----------------------------------------------|---------------------------------------|
| Intent Classification         | chatbots understand the user's goal (intent) | "Book a flight" → booking_intent      |
| Content Moderation            | detect toxic / inappropriate content         | hate speech, harassment detection     |
| Authorship Attribution        | identify who wrote a text                    | plagiarism detection, forensics       |
| Medical Diagnosis             | classify medical notes                       | disease categorization from symptoms  |
| Legal Document Classification | categorize case types                        | contract types, legal domain          |
| Emotion Detection             | fine-grained (detailed) emotion labels       | happy, sad, angry, fearful, surprised |

**Sentiment vs Emotion:** sentiment has 3 broad labels (pos/neg/neutral). Emotion detection is **fine-grained**: angry and fearful are both negative, but they are different emotions.

### Brainstorm (slide 11)

Think (30 s) → Pair (neighbor) → Share (class). Question: "**Which text classification problem would be HARDEST to solve, and why?**" A good answer uses the slide 7 challenges (sarcasm, context, domain language) and/or the number of labels (fine-grained = many close labels).

**Exam traps:** Ham = **not** spam (legitimate). Sentiment on the slide is **3 classes** (multi-class), not binary. Emotion detection ≠ sentiment analysis (more detailed labels).

كل هذه المهام هي نفس الفكرة، والذي يتغير هو قائمة التصنيفات فقط. أصعب شيء في Sentiment Analysis هو sarcasm أي السخرية. و Ham تعني رسالة عادية مشروعة. و Emotion Detection أدق من Sentiment لأنها تفرق بين الغضب والخوف مثلاً.

## Topic 3 — ML Pipeline and Classical Methods (Slides 12-14)

### The Machine Learning Pipeline (slide 12)

A **pipeline** = a fixed order of steps; the output of one step is the input of the next.

| Step        | Slide words      | Meaning                                                                         |
|-------------|------------------|---------------------------------------------------------------------------------|
| 1. Data     | Labeled examples | texts with their correct label (given by humans); a collection = <b>dataset</b> |
| 2. Features | Text → Numbers   | change each text into numbers (clues)                                           |
| 3. Model    | Learn patterns   | find repeating rules from the examples = <b>training</b>                        |

|               |                  |                                           |
|---------------|------------------|-------------------------------------------|
| 4. Prediction | New text → Label | label a text the model has never seen     |
| 5. Evaluation | How good?        | measure how often predictions are correct |

The arrow from 5 goes **back to 1**: the pipeline is a **cycle**. After evaluation, you go back and improve, then repeat.

### Why start with classical methods? (slide 14)

Classical ML = older, simpler methods (before neural networks and transformers). This week: **Naive Bayes, Logistic Regression, SVM**.

| Reason                             | Meaning                                                          |
|------------------------------------|------------------------------------------------------------------|
| Fast to train (seconds to minutes) | learning finishes quickly                                        |
| Easy to interpret and debug        | you can understand <b>why</b> it chose a label and fix mistakes  |
| Work well with small datasets      | no need for millions of examples                                 |
| Low computational requirements     | a normal laptop is enough                                        |
| Strong baselines for comparison    | <b>baseline</b> = simple first model; better models must beat it |

**Worked example (spam pipeline):** Data = 1,000 emails marked SPAM/HAM → Features = count of "!", capital letters, whether "free" appears → Model learns "many ! + free = spam" → Prediction: "FREE gift!!!" → SPAM → Evaluation: 90/100 correct → go back and improve.

**Exam traps:** Training uses **labeled** data; prediction uses **new, unseen** text. Step 2 is where text becomes numbers. Link to the Bulldozer Problem: start simple.

pipeline هو سلسلة خطوات: بيانات معروف تصنيفها، ثم تحويلها إلى أرقام (features)، ثم يتعلم model الأنماط، ثم يتنبأ بنص جديد، ثم نقيم جودته، ونرجع للبداية لنحسن. baseline هو نموذج بسيط نقارن به النماذج الأقوى.

## Topic 4 — Naive Bayes (Slides 15-16)

### Basic terms

- **Probability:** a number from 0 (impossible) to 1 (certain).
- **P(A | B):** "probability of A **given** B" = if we know B is true, how likely is A?

### The Probabilistic Approach (slide 15)

Naive Bayes uses **Bayes' Theorem** to calculate the probability of **each class**, then **picks the class with the highest probability**.

$$P(\text{class} | \text{text}) \propto P(\text{class}) \times P(\text{text} | \text{class})$$

| Part                               | Meaning                                                                        |
|------------------------------------|--------------------------------------------------------------------------------|
| $P(\text{class} \mid \text{text})$ | what we want: given this text, how likely is this class?                       |
| $\propto$                          | "is proportional to": not the exact number, but bigger still means more likely |
| $P(\text{class})$                  | how common the class is in general                                             |
| $P(\text{text} \mid \text{class})$ | if the class is true, how likely is this text?                                 |

Background (not in slides): the full formula also divides by  $P(\text{text})$ , which is the same for every class, so it does not change the winner.

## Why "Naive"?

It **assumes words are independent**: each word is treated alone. This is not true in real language, **but it works surprisingly well**. Slide example: "not good" → "not" and "good" are treated **separately**, so the model misses that "not" flips the meaning.

Because words are independent,  $P(\text{text} \mid \text{class}) = \text{multiply}$  the probability of each word.

| Pros                                | Cons                                   |
|-------------------------------------|----------------------------------------|
| Fast, simple, works with small data | Independence assumption is unrealistic |

## Slide 16 example (spam)

Training data: "Win free money now!" → SPAM; "Click here for prize" → SPAM; "Meeting tomorrow at 3pm" → HAM; "Dinner with family tonight" → HAM.

Learned:  $P(\text{"free"} \mid \text{SPAM}) = \text{high}$ ,  $P(\text{"free"} \mid \text{HAM}) = \text{low}$ ;  $P(\text{"meeting"} \mid \text{HAM}) = \text{high}$ ,  $P(\text{"meeting"} \mid \text{SPAM}) = \text{low}$ .

New text "Win free tickets":

$$P(\text{SPAM} \mid \text{text}) = P(\text{SPAM}) \times P(\text{"win"} \mid \text{SPAM}) \times P(\text{"free"} \mid \text{SPAM}) \times P(\text{"tickets"} \mid \text{SPAM})$$

Calculate for SPAM and HAM, pick the higher → **SPAM**.

## Worked example with numbers (practice numbers, not from the slides)

$P(\text{SPAM}) = 2/4 = 0.5$ ,  $P(\text{HAM}) = 2/4 = 0.5$ .

| Word    | $P(\text{word} \mid \text{SPAM})$ | $P(\text{word} \mid \text{HAM})$ |
|---------|-----------------------------------|----------------------------------|
| win     | 0.3                               | 0.05                             |
| free    | 0.4                               | 0.02                             |
| tickets | 0.1                               | 0.1                              |

1. SPAM:  $0.5 \times 0.3 = 0.15 \rightarrow \times 0.4 = 0.06 \rightarrow \times 0.1 = \mathbf{0.006}$
2. HAM:  $0.5 \times 0.05 = 0.025 \rightarrow \times 0.02 = 0.0005 \rightarrow \times 0.1 = \mathbf{0.00005}$
3.  $0.006 > 0.00005 \rightarrow \mathbf{SPAM}$

**Method:** start with  $P(\text{class}) \rightarrow$  multiply by  $P(\text{word} \mid \text{class})$  for every word  $\rightarrow$  do it for each class  $\rightarrow$  pick the biggest.

**Exam traps:** "Naive" = the **independence assumption** (not "simple code"). A text with a spam word can still be HAM if the other words are strongly HAM — all words count. Negation ("not good", "not bad") is the classic NB failure.

فكرة Naive Bayes: نحسب لكل class رقما، وهو مدى شيوع class مضروبا في احتمال كل كلمة، ثم نختار الأكبر. وسبب تسميته Naive أنه يفترض أن الكلمات independent أي مستقلة عن بعضها، وهذا غير واقعي، لذلك لا يفهم عبارة مثل not good.

## Topic 5 — Logistic Regression (Slides 18-20)

### "Linear Classification with Probabilities" (slide 18)

**Despite the name, it is a CLASSIFICATION algorithm!** (Background, not in slides: "regression" usually means predicting a number, which is why the name confuses people.)

#### Terms

- $x_1, x_2, \dots$  = features (e.g.  $x = 1$  if a word is in the text, 0 if not).
- $w_1, w_2, \dots$  = **weights**: a positive weight pushes toward class 1, a negative weight pushes toward class 0, and a bigger size means a stronger effect.
- $b$  = **bias**: a starting number that is always added.
- $y = 1$  = the text belongs to class 1 (e.g. SPAM).
- $\sigma$  (**sigmoid**) = a function that squeezes any number into **0 to 1**: big positive  $\rightarrow$  near 1, 0  $\rightarrow$  0.5, big negative  $\rightarrow$  near 0.

#### How it works (3 steps)

1. Learn **weights** for each feature (word).
2. Calculate the **weighted sum**.
3. Apply the **sigmoid function** to get a probability (0-1).

$$P(y=1) = \sigma(w_1x_1 + w_2x_2 + \dots + b)$$

Background (not in slides): usually, if  $P(y=1) > 0.5$ , the model chooses class 1.

| Pros                                              | Cons                                      |
|---------------------------------------------------|-------------------------------------------|
| Fast; interpretable weights; probabilistic output | Assumes a <b>linear decision boundary</b> |

**Decision boundary** = the border that separates the classes. **Linear** = a straight line. If the classes cannot be separated by a straight line, LR has trouble.

#### Key difference from Naive Bayes

|      | Naive Bayes       | Logistic Regression   |
|------|-------------------|-----------------------|
| Type | <b>Generative</b> | <b>Discriminative</b> |

|        |                                                                                                        |                                                         |
|--------|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| Learns | <b>models distributions:</b> how the texts of each class look, i.e. $P(\text{word} \mid \text{class})$ | <b>learns the boundary</b> between the classes directly |
|--------|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------|

## Quick Poll (slide 20)

When would you use NB over LR? **Answer A: when you have VERY small training data and need fast training.** (Not B: "absolute best accuracy regardless of speed".) NB shines with small data and when speed matters.

## Worked example (practice numbers)

Weights: free = +2, win = +1.5, meeting = -2, b = -1. Text "Win free tickets":

1.  $x_{\text{free}} = 1, x_{\text{win}} = 1, x_{\text{meeting}} = 0$
2.  $\text{Sum} = 2 + 1.5 + 0 - 1 = \mathbf{2.5}$
3.  $\sigma(2.5) \approx \mathbf{0.92}$
4.  $P(\text{SPAM}) \text{ high} \rightarrow \mathbf{SPAM}$

Reading the weights: "free" (+2) is the strongest spam word, and "meeting" (-2) is the strongest ham word. This is "interpretable weights".

**Exam traps:** LR is classification, not regression. NB = generative, LR = discriminative (students often swap these). The con of LR is the **linear** boundary — SVM's kernel trick solves this.

رغم اسمه، Logistic Regression خوارزمية classification. لكل كلمة weight، نجمعها مع b، ثم نستخدم sigmoid لنحصل على احتمال بين 0 و 1. Naive Bayes هو generative أي يتعلم شكل نصوص كل class، أما Logistic Regression فهو discriminative أي يتعلم الحد الفاصل مباشرة، وعيبه أن هذا الحد خط مستقيم.

## Topic 6 — Support Vector Machines (Slide 22)

### "Finding the Best Boundary"

SVM finds the **hyperplane** that **maximizes the margin** between classes.

- **Hyperplane** = the boundary: a line (2 features), a flat surface (3 features), and so on.
- **Dimension** = one feature (2 features = 2 dimensions).
- **Margin** = the distance between the boundary and the closest points of each class.

**Key idea:** maximize the margin. The decision boundary should be **as far as possible from both classes**. Many lines can separate the classes, and SVM picks the one with the biggest margin.

|                      |                             |
|----------------------|-----------------------------|
| Small margin (bad)   | Large margin (SVM's choice) |
| H H   S S            | H H :   : S S               |
| H H   S              | H H :   : S                 |
| ^ touches both sides | <-margin->                  |

## The Kernel Trick

**Transform the data to higher dimensions where it is linearly separable. Linearly separable** = a straight line can separate the classes. **Kernel** = the function that does the transformation. Kernels named: **Linear, RBF, polynomial**.

| Pros                                                                 | Cons                         |
|----------------------------------------------------------------------|------------------------------|
| Effective in high dimensions (many features — text has one per word) | Slow for large datasets      |
| Memory efficient                                                     | Sensitive to feature scaling |
| Versatile (kernels)                                                  |                              |

**Feature scaling:** features with big numbers (word count = 800) can dominate features with small numbers (capital ratio = 0.2). Scaling = putting features in similar ranges.

### Worked example: kernel trick

Class A:  $x = -3, -2, 2, 3$ . Class B:  $x = -1, 0, 1$ .

1. On one line, A is on **both sides** of B, so no single cut works.
2. Add the feature  $x^2$ : A  $\rightarrow 9, 4, 4, 9$ ; B  $\rightarrow 1, 0, 1$ .
3.  $A \geq 4$  and  $B \leq 1$ , so a straight line at  $x^2 = 2.5$  separates them  $\rightarrow$  **linearly separable**.

**Exam traps:** the kernel trick goes to **higher** dimensions (not lower). SVM = most accurate but **slow** on big data. SVM fixes LR's "linear boundary" problem through kernels.

SVM يختار الخط الذي يكون أبعد ما يمكن عن الطرفين، وهذه المسافة اسمها margin. وعندما لا يستطيع خط مستقيم أن يفصل البيانات، نستخدم Kernel Trick لننقل البيانات إلى أبعاد أعلى حتى تصبح linearly separable. عيوبه: بطيء مع البيانات الكبيرة، ويتأثر بمقاييس features.

## Topic 7 — Feature Engineering (Slide 24)

**Feature engineering** = designing and choosing good features from text. "**Good features = Good performance.**" Classical ML **relies heavily on feature quality**.

| Type                               | Features on the slide                                                              | Meaning                             |
|------------------------------------|------------------------------------------------------------------------------------|-------------------------------------|
| <b>Statistical</b>                 | text length, word count; avg word length; punctuation counts; capital letter ratio | simple numbers you count or measure |
| <b>Lexical</b> (about words)       | Bag of Words; TF-IDF; N-grams (bigrams, trigrams); character n-grams               | features about the words themselves |
| <b>Linguistic</b> (about language) | POS tags; sentiment lexicons; named entities; dependency patterns                  | grammar and meaning                 |

|                        |                                                                                                     |                             |
|------------------------|-----------------------------------------------------------------------------------------------------|-----------------------------|
| <b>Domain-specific</b> | Spam: "free", "click", "!!!";<br>Sentiment: emoticons; Medical:<br>terminology; custom dictionaries | from knowledge of the field |
|------------------------|-----------------------------------------------------------------------------------------------------|-----------------------------|

## Term definitions

- **Bag of Words:** count each word and **ignore word order**.
- **TF-IDF:** a word's score is high when the word is frequent in **this** text but **rare** in other texts (TF = term frequency, IDF = inverse document frequency).
- **N-gram:** n words next to each other. Bigram = 2, trigram = 3. **Character n-gram** = n letters.
- **POS tags:** Part-Of-Speech (noun, verb, adjective...).
- **Sentiment lexicon:** a word list with sentiments (excellent = +, awful = -).
- **Named entities:** names of people, places, companies.
- **Dependency patterns:** grammar links between words (which word depends on which).

**Pro tip (slide):** for projects, **combining multiple feature types often works best**.

## Worked example: statistical features (slide 8 texts)

| Feature         | "CONGRATULATIONS! You've WON \$1000000!!!" | "Meeting tomorrow at 3pm" |
|-----------------|--------------------------------------------|---------------------------|
| Word count      | 4                                          | 4                         |
| "!" count       | 4                                          | 0                         |
| Letters         | 15 + 5 + 3 = 23                            | 7 + 8 + 2 + 2 = 19        |
| Capital letters | 15 + 1 + 3 = 19                            | 1                         |
| Capital ratio   | 19/23 $\approx$ 0.83                       | 1/19 $\approx$ 0.05       |

Word count does not help here, but "!" count and capital ratio separate spam from ham clearly.

## Worked example: n-grams

"the food was not good" → bigrams: the food, food was, was not, **not good**. Trigrams: the food was, food was not, was not good. Now "not good" is **one feature**, which fixes the Naive Bayes "not good" problem.

**Exam traps:** emoticons = **Domain-specific** (sentiment), but sentiment lexicons = **Linguistic**. Bag of Words **loses word order**. TF-IDF and n-grams are **Lexical**; POS and named entities are **Linguistic**.

Feature engineering هو اختيار الأرقام التي تصف النص، لأن classical ML يعتمد بشدة على جودتها. الأنواع الأربعة هي: Statistical وهي أرقام بسيطة، و Lexical وهي عن الكلمات، و Linguistic وهي عن القواعد والمعنى، و Domain-specific وهي من معرفة المجال. و bigram مثل not good يحل مشكلة Naive Bayes.

## Topic 8 — Comparing Methods, Project 1, Takeaways (Slides 25-31)

### Comparing Methods (slide 25) — more = better

| Method              | Speed | Small Data | Interpretability | Accuracy | Best For                                |
|---------------------|-------|------------|------------------|----------|-----------------------------------------|
| Naive Bayes         | 3/3   | 3/3        | 2/3              | 2/4      | spam, quick baselines                   |
| Logistic Regression | 2/3   | 2/3        | 3/3              | 3/4      | sentiment, general purpose              |
| SVM                 | 1/3   | 1/3        | 1/3              | 4/4      | when accuracy matters, smaller datasets |

(On the slide: lightning bolts = speed, check marks = small data / interpretability, stars = accuracy.)

**For your project:** start with **Naive Bayes as baseline** → try **Logistic Regression** → if time permits, try **SVM**. **Next week:** neural methods that can beat all of these.

### Project 1: Text Classification (slide 26)

| Item         | Requirement                                                                         |
|--------------|-------------------------------------------------------------------------------------|
| Team         | 5 students                                                                          |
| Task         | choose ONE classification problem (sentiment, spam, topic, etc.)                    |
| Methods      | at least <b>Naive Bayes + one other method</b> from class                           |
| Evaluation   | compare your results with <b>benchmarks</b> (known results on the same data)        |
| Presentation | 3 minutes + 2 minutes Q&A (Week 4)                                                  |
| Timeline     | Week 2: form teams, choose problem / Week 3: implementation / Week 4: presentations |
| Deadline     | <b>GitHub repo due before your presentation</b>                                     |

### Evaluation Rubric (slide 27)

| Part | Weight | Graded items |
|------|--------|--------------|
|------|--------|--------------|

|                          |            |                                                                                                                                                                                                           |
|--------------------------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Technological Depth      | <b>40%</b> | code quality and implementation correctness; number of methods compared (more = better depth); feature engineering creativity; proper evaluation (train/test split, metrics); error analysis and insights |
| Idea Originality         | <b>10%</b> | creative problem selection or dataset choice; novel feature combinations; interesting domain application                                                                                                  |
| Presentation Performance | <b>50%</b> | clear explanation of problem and approach; effective communication of technical concepts; professional slides and delivery; handling Q&A confidently; time management (stay within 3 minutes)             |

- **Train/test split:** train on one part of the data and test on another part the model has never seen.
- **Metrics:** numbers that measure quality (e.g. accuracy).
- **Error analysis:** studying the model's mistakes to understand why they happen.

### Some Ideas (slide 28)

| Area               | Ideas                                                                      |
|--------------------|----------------------------------------------------------------------------|
| Sentiment          | movie, product, restaurant reviews; social media posts; app store reviews  |
| Spam               | email, SMS, comment, social media spam                                     |
| Topic              | news categorization; research paper classification; support ticket routing |
| Content Moderation | toxic comment, hate speech, cyberbullying detection                        |
| Emotion            | joy, anger, sadness, fear; customer service tones                          |
| Domain-Specific    | medical notes; legal document types; financial news sentiment              |

Be creative: find a dataset or application that excites your team.

### Where to Find Data (slide 29)

| Source      | URL                                                                   | Datasets                                            |
|-------------|-----------------------------------------------------------------------|-----------------------------------------------------|
| HuggingFace | <a href="https://huggingface.co/datasets">huggingface.co/datasets</a> | IMDB, Amazon, AG News, Yelp, Tweet sentiment, SST-2 |

|                   |                        |                                                                         |
|-------------------|------------------------|-------------------------------------------------------------------------|
| Kaggle            | kaggle.com/datasets    | spam classification, news categorization, sentiment140, disaster tweets |
| UCI ML Repository | archive.ics.uci.edu/ml | SMS Spam Collection, YouTube spam, Reuters news                         |

**Important:** at least **1,000 examples**, **clear labels**, and **check the license** (legal permission to use the data).

### Key Takeaways (slide 30)

1. Text classification = assigning labels to documents
2. Many applications: sentiment, spam, topics, content moderation
3. Classical methods: Naive Bayes (**fast**), Logistic Regression (**balanced**), SVM (**accurate**)
4. Feature engineering is crucial for classical ML
5. Project: build a classifier, compare methods, present findings

**Start with simple baselines, then improve!**

### What's Next (slide 31)

Week 3: neural methods for text classification — word embeddings (Word2Vec, GloVe), CNNs for text (convolutions on sequences), RNNs and BERT (modern approaches). Before next week: form a team of 5, choose a problem, find and explore a dataset, set up a GitHub repo.

**Exam traps:** the most **interpretable** method is **Logistic Regression**, not Naive Bayes. Presentation = **50%** (the biggest part). NB is **required** in the project.

Naive Bayes سريع، و Logistic Regression متوازن والأسهل للفهم، و SVM هو الأدق لكنه الأبطأ. في المشروع يجب استخدام Naive Bayes مع طريقة أخرى، والعرض يساوي 50 بالمئة من الدرجة ولا يتجاوز 3 دقائق، والبيانات يجب أن تحتوي على 1000 مثال على الأقل.

## Formula Sheet

| Formula                                                                                                              | Where               | Read it as                                                                          |
|----------------------------------------------------------------------------------------------------------------------|---------------------|-------------------------------------------------------------------------------------|
| $f(\text{text}) \rightarrow \text{label}$                                                                            | Topic 1             | a classifier turns a text into one label                                            |
| $P(\text{class}   \text{text}) \propto P(\text{class}) \times P(\text{text}   \text{class})$                         | Naive Bayes         | class score = how common the class is $\times$ how likely the text is in this class |
| $P(\text{SPAM}   \text{text}) = P(\text{SPAM}) \times P(w_1   \text{SPAM}) \times P(w_2   \text{SPAM}) \times \dots$ | Naive Bayes         | multiply word probabilities (independence)                                          |
| $P(y=1) = \sigma(w_1x_1 + w_2x_2 + \dots + b)$                                                                       | Logistic Regression | sigmoid of (weighted sum + bias)                                                    |
| Capital ratio = capital letters $\div$ all letters                                                                   | Feature engineering | a statistical feature                                                               |

## Glossary (A-Z)

| Term                  | Meaning                                    |
|-----------------------|--------------------------------------------|
| Accuracy              | how often the model is correct             |
| Bag of Words          | word counts, word order ignored            |
| Baseline              | simple first model for comparison          |
| Benchmark             | known results to compare against           |
| Bias (b)              | number always added in Logistic Regression |
| Binary classification | exactly 2 labels                           |
| Classical ML          | older, simpler methods: NB, LR, SVM        |
| Decision boundary     | the border separating classes              |
| Dimension             | one feature / one direction                |
| Discriminative        | learns the boundary (LR)                   |
| Domain                | a field or area (medical, legal...)        |
| Error analysis        | studying the model's mistakes              |
| Feature               | a number/clue describing a text            |
| Feature engineering   | designing good features                    |
| Feature scaling       | putting features in similar ranges         |
| Generative            | models how each class's texts look (NB)    |
| Ham                   | legitimate, non-spam message               |

|                      |                                                              |
|----------------------|--------------------------------------------------------------|
| Hyperplane           | the boundary (line, plane, ...)                              |
| Intent               | the user's goal (chatbots)                                   |
| Interpretability     | how easy it is to see why a label was chosen                 |
| Kernel trick         | move data to higher dimensions to make it linearly separable |
| Label / class        | the name of a category                                       |
| Linearly separable   | a straight line can separate the classes                     |
| Margin               | distance from the boundary to the closest points             |
| Metrics              | numbers that measure model quality                           |
| Multi-class          | more than 2 labels                                           |
| N-gram               | n neighboring words (bigram = 2, trigram = 3)                |
| Named entities       | names of people, places, companies                           |
| Pipeline             | fixed order of steps                                         |
| POS tags             | parts of speech (noun, verb, adjective)                      |
| Predefined           | fixed in advance                                             |
| Sarcasm              | words mean the opposite                                      |
| Sentiment            | feeling/opinion in text                                      |
| Sentiment lexicon    | word list with sentiments                                    |
| Sigmoid ( $\sigma$ ) | squeezes any number into 0-1                                 |
| TF-IDF               | high for words frequent here but rare elsewhere              |
| Train/test split     | train on one part, test on an unseen part                    |
| Training             | the model learning from labeled data                         |
| Weight (w)           | importance and direction of a feature                        |

## Self-Test (answers on the next page)

### Part A — Multiple choice

1. Text classification means: (a) generating new text (b) assigning predefined labels to text (c) translating text (d) summarizing text
2. "Sports, Politics, Tech, Business" is: (a) binary (b) multi-class (c) sentiment (d) intent
3. Ham means: (a) spam (b) a toxic message (c) a legitimate message (d) a neutral review
4. Why is Naive Bayes "naive"? (a) its code is short (b) it assumes words are independent (c) it needs little data (d) it uses probabilities
5. Which is discriminative? (a) Naive Bayes (b) Logistic Regression (c) Bag of Words (d) TF-IDF
6. The con of Logistic Regression is: (a) slow for large data (b) independence assumption (c) assumes a linear decision boundary (d) sensitive to emoticons
7. The kernel trick: (a) moves data to lower dimensions (b) removes features (c) moves data to higher dimensions to make it linearly separable (d) maximizes word count
8. The most interpretable method (slide 25) is: (a) Naive Bayes (b) Logistic Regression (c) SVM (d) all are equal
9. "Emoticons" are which feature type? (a) Statistical (b) Lexical (c) Linguistic (d) Domain-specific
10. The largest part of the project rubric is: (a) Technological Depth (b) Idea Originality (c) Presentation Performance (d) Dataset size

### Part B — Open questions

1. Naive Bayes,  $P(\text{SPAM}) = P(\text{HAM}) = 0.5$ . Text "free meeting tonight".  $P(\text{free}|\text{SPAM}) = 0.4$ ,  $P(\text{meeting}|\text{SPAM}) = 0.02$ ,  $P(\text{tonight}|\text{SPAM}) = 0.05$ ;  $P(\text{free}|\text{HAM}) = 0.02$ ,  $P(\text{meeting}|\text{HAM}) = 0.3$ ,  $P(\text{tonight}|\text{HAM}) = 0.2$ . Which class wins?
2. Logistic Regression ( $y = 1 = \text{Positive}$ ): great = +2, terrible = -3, boring = -1.5,  $b = +0.5$ .  $\sigma(1) \approx 0.73$ ,  $\sigma(-4) \approx 0.02$ . Classify "great but boring" and "terrible and boring".
3. Name the 5 steps of the ML pipeline and explain why the arrow goes from step 5 back to step 1.
4. Explain why "not bad" is a problem for Naive Bayes, and which feature from slide 24 helps.
5. A team of 4 uses only Logistic Regression and SVM, on a dataset of 500 examples with no license. List every rule they break.

## Self-Test — Answers

### Part A

| Q  | Answer | Why                                                                             |
|----|--------|---------------------------------------------------------------------------------|
| 1  | b      | definition on slide 6; (a), (c), (d) produce new text, not a label              |
| 2  | b      | more than 2 labels                                                              |
| 3  | c      | ham = legitimate (not spam)                                                     |
| 4  | b      | independence assumption; (c) is a pro, not the reason for the name              |
| 5  | b      | LR learns the boundary; NB is generative                                        |
| 6  | c      | (a) is SVM's con, (b) is NB's con                                               |
| 7  | c      | higher, not lower                                                               |
| 8  | b      | LR has 3/3 interpretability; NB only 2/3 (common trap)                          |
| 9  | d      | slide lists emoticons under Domain-specific (sentiment lexicons are Linguistic) |
| 10 | c      | 50% (Depth 40%, Originality 10%)                                                |

### Part B

**1.** SPAM:  $0.5 \times 0.4 = 0.2 \rightarrow \times 0.02 = 0.004 \rightarrow \times 0.05 = \mathbf{0.0002}$ . HAM:  $0.5 \times 0.02 = 0.01 \rightarrow \times 0.3 = 0.003 \rightarrow \times 0.2 = \mathbf{0.0006}$ . HAM wins, because "meeting" and "tonight" together beat "free".

**2.** "great but boring":  $2 - 1.5 + 0.5 = 1 \rightarrow \sigma(1) \approx 0.73 \rightarrow \mathbf{Positive}$ . "terrible and boring":  $-3 - 1.5 + 0.5 = -4 \rightarrow \sigma(-4) \approx 0.02 \rightarrow \mathbf{Negative}$ .

**3.** Data (labeled examples)  $\rightarrow$  Features (text  $\rightarrow$  numbers)  $\rightarrow$  Model (learn patterns)  $\rightarrow$  Prediction (new text  $\rightarrow$  label)  $\rightarrow$  Evaluation (how good?). The arrow back means the pipeline is a **cycle**: after evaluating, you return to improve (e.g. more or better data) and repeat.

**4.** NB treats words as independent, so it sees "bad" (negative) and ignores that "not" flips it. **N-grams (bigrams)** make "not bad" one feature (Lexical features).

**5.** (1) The team must have 5 students. (2) The project must include Naive Bayes. (3) The dataset needs at least 1,000 examples. (4) They must check the license.